

Le Rapporteur: Fail-Closed, Database-Verified Legal Question Answering on Sovereign Hardware

Wilfred Doré

Équipe Le Rapporteur

Hackathon Assemblée nationale, France

wilfred.dore@telecom-paristech.org

François Amat

Équipe Le Rapporteur

Hackathon Assemblée nationale, France

contact@famam.me

ABSTRACT

Large language models (LLMs) answer legal questions fluently, but they routinely *invent* statutory citations that look plausible and are wrong. In law, a confident wrong answer is worse than no answer. We present *Le Rapporteur*, a legal question-answering system built on one non-negotiable rule: **no verifiable source, no answer**. The model generates a draft; every cited article (e.g., *article L. 3121-27 du Code du travail*) is extracted and checked, one by one, against a live legal database (Canutes, a mirror of the French Légifrance corpus); if a single citation cannot be resolved to an *in-force* article, the system returns an explicit refusal rather than a fabrication. We describe (i) the database-centric verifier and the query engineering required to check citations at interactive latency over a 253-table corpus without full-table scans, (ii) a reusable trust service exposing the verifier over HTTP and the Model Context Protocol (MCP) so any agent can fact-check an LLM’s output, (iii) a hardware-sovereign serving layer that runs the identical pipeline on non-NVIDIA silicon (Qualcomm Cloud AI 100, live) via a single OpenAI-compatible indirection, and (iv) an executable, jurist-readable trust specification in Gherkin where one fabricated article breaks the build. On a suite of 100 citizen legal questions run live, every citation surfaced as verified resolved to an in-force article in the database; we also report a live model that cited a plausible but wrong article, caught by the verifier, and discuss the gap between *existence* checking and *semantic* correctness, which motivates our grounding roadmap.

KEYWORDS

legal NLP, retrieval, hallucination, fact verification, LLM serving, sovereignty, Model Context Protocol, open government data

Draft — PVLDB-style formatting. Artifacts: <https://github.com/wilfred-dore/hackathon-assemblée-2026>.

1 INTRODUCTION

Citizens increasingly turn to general-purpose large language models (LLMs) for questions of law: *Am I entitled to a year-end bonus? What is the legal working week? Can my landlord keep the deposit?* The models reply in fluent, authoritative prose—and sometimes cite articles that do not exist, that exist but have been repealed, or that exist and are in force yet say something entirely different from what the answer claims. In open-domain conversation such slips are tolerable; in law they are not. The domain has an *asymmetric* cost structure: a confident, plausible-sounding, fabricated citation actively *misleads* a citizen who cannot easily distinguish it from a real one, and is therefore strictly worse than an honest admission of ignorance. Recent work profiling *legal* hallucinations

reports that general-purpose LLMs invent or misattribute legal authorities at high rates [3], confirming that the problem is systemic rather than anecdotal.

Hallucination in natural language generation is by now well documented [7, 8], and retrieval-augmented generation (RAG) is the standard mitigation [5, 10]: the model is conditioned on retrieved passages so that its output is, ideally, grounded in real text. Yet grounding *reduces* but does not *eliminate* invented references. A model handed the correct passage can still transpose a digit in an article number, cite a neighbouring article, or attribute a real quotation to the wrong code. Grounding improves the *expected* faithfulness of a draft; it does not *guarantee* any individual citation. In law, the guarantee is what matters.

Our stance: refusing is a feature. *Le Rapporteur* adopts a deliberately conservative position suited to the domain, organized around a single, non-negotiable invariant: **no verifiable source, no answer**. The invariant is enforced not by prompt engineering—advisory, and easily undone by the model’s own sampling—but by a *post-generation verification stage* that is *fail-closed*. The model drafts freely; but before anything reaches the user, every citation in the draft is checked, one by one, against an authoritative legal database. If a single citation cannot be resolved to a real, in-force article, or if any error occurs while checking (a network timeout, a malformed reference, an ambiguous code), the system does not guess: it returns an explicit refusal—*“Je ne trouve pas de texte applicable.”* (*I find no applicable text*). Doubt collapses to refusal, never to fabrication. This is defense in depth: the trust guarantee is a property of the *system* and holds even when the model misbehaves.

A running example. Two questions recur throughout the paper. Asked *“Do I have a right to a Christmas bonus?”*, a fluent model confidently invents *article L. 4321-7 du Code du travail*—an authority that simply does not exist. Asked for *“the legal working week”*, a live model we tested answered *article L. 3132-2* (which exists, but governs weekly *rest*) instead of the correct *L. 3121-27*. The first is a non-existent authority; the second is real-but-wrong. A system that merely pattern-matches “looks like a citation” passes both; *Le Rapporteur* refuses the first outright and, as we discuss (§9, §10), exposes the second as the harder, semantic case.

Contributions.

- A **fail-closed pipeline** for legal QA—*generate, extract, verify, source-or-refuse*—whose trust guarantee is a property of the *system*, not of the model, stated as a precise release predicate (§3).
- A **database-centric citation verifier** over the Canutes/Légifrance corpus, and the query engineering—index-friendly variant expansion, accent-folded code disambiguation, state-aware

selection of the in-force version—that makes per-citation checking feasible at interactive latency over a large, irregular schema without full-table regular-expression scans (§4).

- A **reusable trust service**: the verifier is exposed over HTTP (/verify) and as a Model Context Protocol (MCP) server, so any third-party agent can fact-check the citations in an arbitrary LLM output without adopting our model (§5).
- A **hardware-sovereign serving layer**: a thin OpenAI-compatible indirection lets the identical pipeline run on non-NVIDIA accelerators; we demonstrate it live on a Qualcomm Cloud AI 100 and target AMD via a portable runtime (§6).
- An **executable trust specification** in Gherkin, readable and auditable by a jurist, in which a single fabricated article turns the build red (§7).

Scope and honesty. This system was built under hackathon time constraints. We are deliberately candid about what is *proven* (runs today, tested) versus *envisioned* (designed, on the roadmap), and we mark the boundary explicitly rather than blur it—an editorial choice mirroring the system’s own fail-closed ethos. The remainder of the paper follows the pipeline: §2 frames the corpus and a taxonomy of citation errors; §3 presents the pipeline and its release predicate; §4 details the database verifier that is the technical core; §5 the reusable trust service; §6 the sovereign serving layer; §7 the executable specification; and §§9–10 our observations and the road from existence checking to semantic verification.

2 BACKGROUND AND PROBLEM

2.1 French legal open data

French statutory law is published as open data by the DILA (*Direction de l’information légale et administrative*) through Légifrance [4]. The corpus is vast—dozens of codes, each with thousands of articles, alongside decrees, ordinances, and case law—and, crucially, it is *living*: articles are created, amended, and repealed continuously, and every change produces a new version while prior versions are retained for the historical record. Our verification target is *Canutes*, a PostgreSQL corpus mirroring this data. Two structural properties dominate the engineering.

The schema is large and irregular. Canutes contains on the order of 253 tables with terse, historically accreted names, and identifiers follow Légifrance’s own conventions (LEGITEXT... for a text, LEGIARTI... for an article version, LEGISCTA... for a section). A public PostgREST [13] endpoint exposes only three of these tables—enough to browse, not to verify. Sound verification needs schema-aware access to the full article table rather than the sanitized public view, and cannot assume a clean star schema or documented foreign keys.

Articles are versioned and stateful. The same numbered article—say *L. 3121-27*—appears as *several rows*, one per historical version, each carrying a state (*étatArticle*) such as *VIGUEUR* (in force), *ABROGE* (repealed), or *MODIFIE* (superseded), and a validity interval (*dateDebut*, *dateFin*). For a citizen’s question only the *VIGUEUR* version is authoritative: presenting a repealed version as current is itself a form of hallucination, even though the article “exists”. Verification must therefore be *state-aware*, not a bare existence lookup.

Table 1: A taxonomy of legal citation errors. A state-aware existence check catches E1–E3; E4 requires semantic verification.

Type	Failure mode	Caught by existence?
E1	Article number does not exist	yes
E2	Exists, but attributed to the wrong code	yes
E3	Exists, but repealed (state $\neq$ VIGUEUR)	yes
E4	Exists, in force, but off-topic	no (semantic)

2.2 A taxonomy of citation errors

Stating the problem precisely clarifies what a verifier must catch. Table 1 distinguishes four failure modes by *where* a citation departs from ground truth. A state-aware existence check—our contribution—catches the first three cleanly; the fourth (real, in force, but semantically off-topic) is the hard case that motivates our roadmap (§10).

2.3 Why not RAG alone

Retrieval-augmented generation [5, 10] conditions the model on retrieved passages and measurably improves faithfulness, and dense retrievers [9] make such conditioning practical at scale. But RAG shifts the error distribution without bounding it: a model can paraphrase a correctly retrieved passage into a wrong number (E1), substitute a neighbouring article, or attribute the right text to the wrong code (E2). Self-consistency methods such as SelfCheckGPT [11] sample the model against itself, but a confidently wrong citation is frequently *self-consistent* and thus invisible to them. Our position is that grounding and *post-hoc verification against an external source of truth* are complementary: grounding improves the draft in expectation; verification provides a per-citation guarantee. This paper focuses on the guarantee and treats the model as an untrusted component.

2.4 Anatomy of an article record

In Canutes each article version is, in essence, a row (*id*, *num*, *data*) whose data payload (JSON) carries both metadata and text: the state (*VIGUEUR*, *ABROGE*, ...), the validity interval, the article’s textual content, and a reference to the parent text (*LEGITEXT*) that identifies the *code*. One consequence matters for verification: the code name lives *inside* the JSON payload rather than in a normalized, indexed column, so disambiguating *which* code an article belongs to is a text match against the payload rather than a foreign-key join—directly motivating the two-stage filter of §4.

2.5 Threat model

We separate *benign* error—a fluent model confidently misremembering, the dominant source of legal misinformation reaching ordinary citizens—from *adversarial* manipulation. Le Rapporteur is designed against benign error and is robust to it *by construction*: no sampling of the model can make the verifier accept a non-existent or repealed article, because acceptance is decided by the database, not the model. We do *not* claim robustness against an adversary who controls the legal database or the retrieval channel; those—the database and the verifier code—constitute the trusted base (§3).

Algorithm 1 Fail-closed answering. A single unverifiable citation—or any exception raised while checking—yields a refusal.

```

1: function ANSWER( $q$ )
2:    $a \leftarrow \text{LLM}(q)$  ▷ untrusted draft
3:    $C \leftarrow \text{EXTRACTCITATIONS}(a)$ 
4:   if  $C = \emptyset$  then
5:     return REFUSE
6:   end if
7:    $S \leftarrow \emptyset$ 
8:   for  $c \in C$  do
9:      $r \leftarrow \text{VERIFY}(c)$  ▷ state-aware DB lookup
10:    if  $\neg r.\text{exists}$  then
11:      return REFUSE ▷ fail-closed
12:    end if
13:     $S \leftarrow S \cup \{\langle c, r.\text{url}, r.\text{excerpt} \rangle\}$ 
14:  end for
15:  return SOURCED( $a, S$ )
16: end function

```

3 SYSTEM OVERVIEW

Figure 1 shows the four-stage pipeline—*generate*, *extract*, *verify*, *source-or-refuse*—and Algorithm 1 states it precisely. A citizen question q is sent to a sovereign LLM (§6), which returns a draft answer a . The draft is parsed for legal citations of the canonical French form “*article* $\langle \text{num} \rangle$ *du* $\langle \text{code} \rangle$ ” by a normalization-tolerant extractor (§4). Each extracted citation c_i is verified *independently* against the legal database, yielding a verdict together with, on success, a canonical LEGIARTI identifier, a clickable Légifrance URL, and a short excerpt.

The release predicate. Let $C(a) = \{c_1, \dots, c_n\}$ be the citations extracted from draft a . The answer is released to the user *if and only if*

$$C(a) \neq \emptyset \wedge \forall c_i \in C(a) : \text{verify}(c_i) = \text{OK}.$$

Every other outcome—no citation at all, one unresolved citation, or any exception while checking—maps to an explicit refusal. The disjunction is deliberately lopsided: the release branch demands *universal* success, whereas a *single* failure anywhere triggers refusal. This is what we mean by *fail-closed*, and it is the whole point: the guarantee does not depend on the model behaving well, only on the verifier being sound about what it can confirm.

Why independent, per-citation checks. Verifying citations one at a time—rather than scoring the answer as a whole—keeps the trusted computing base small and its verdicts explainable: each released citation is accompanied by the exact identifier and URL that justify it, so a user (or an auditor) can click through to the primary source. Independence also makes the layer trivially parallelizable and lets a partial failure localize to the offending citation instead of tainting the whole answer.

3.1 Trusted computing base

The components that must be correct for the guarantee to hold are few and explicit: the citation extractor, the verifier, and the legal database. The model, the network, the serving hardware, and even the prompt lie *outside* the trusted base—any may fail arbitrarily,

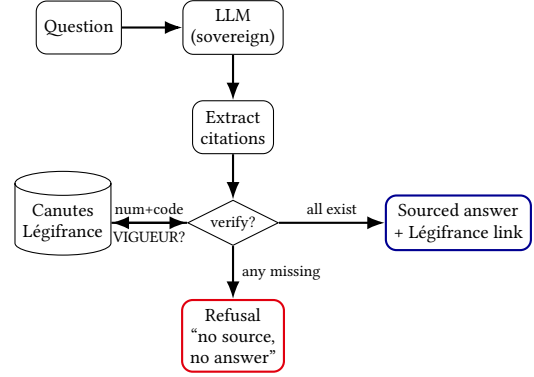


Figure 1: The fail-closed pipeline: *generate*, *extract*, *verify*, *source-or-refuse*. The trust guarantee is enforced at the verification diamond, independently of the model.

and the worst observable outcome is a refusal, never a fabricated citation presented as verified. Shrinking the trusted base to a database lookup plus a few hundred lines of verification code is what makes the guarantee auditable rather than aspirational.

3.2 Where the latency goes

End-to-end latency is dominated by *generation*, not verification: the verifier issues a handful of bounded, index-friendly queries (§4) whose cost is negligible beside a multi-hundred-token decode. Per-citation checking is therefore affordable online, and the end-to-end latency we report (§9) tracks the model rather than the database—verification buys the guarantee essentially for free relative to the cost already paid to generate the draft.

4 CITATION VERIFICATION OVER A LEGAL DATABASE

The verifier answers a deceptively simple question: *does article* $\langle \text{num} \rangle$ *of* $\langle \text{code} \rangle$ *exist, and is it in force?* Answering it soundly *and* at interactive latency over Canutes is the technical core of the system.

4.1 Finding the needle in 253 tables

Because Canutes ships without documented foreign keys and with opaque, historically accreted table names, *locating* the authoritative article table is itself work. We recover a usable map of the schema *offline*—entity-and-relationship inference over the 253 tables, cross-checked against Légifrance’s identifier conventions (LEGITEXT/LEGIARTI) and pin verification to `legifrance.article`, whose rows are the versioned records of §2. Paying this cost once, offline, keeps the online path a single, well-understood query rather than an exploratory crawl across an unknown schema.

4.2 Extraction and normalization

Citations are first lifted from the draft by a tolerant matcher keyed on the French legal idiom (“*article* ... *du* *Code* ...”), capturing the article number and the code name. Légifrance, however, encodes article numbers *inconsistently*: the same article surfaces as L3121–27, L. 3121–27, or L. 3121–27, with or without a space after the book

Algorithm 2 Variant expansion: turn one written citation number into a closed set of exact strings for index-friendly equality matching.

```

1: function VARIANTS(num)
2:   core ← strip spaces and periods from num ▶ e.g. L3121-27
3:   return the closed set re-inserting the optional period and
      space after the leading letter(s), e.g. {L3121-27, L. 3121-27, L.
      3121-27, L 3121-27}
4: end function

```

letter and with or without the period. A naive existence predicate that tries to absorb this variation with a pattern match—`num ~ '.*3121-27.*'`—forces a sequential scan and a regular-expression evaluation over the *entire* `legifrance.article` table; in our environment this reliably hit the statement timeout.

Instead, we push the variation to the *client*: a citation is expanded into a small, closed set of *exact* string variants (Algorithm 2), which are then matched with *equality* against the indexable `num` column. Equality against a bounded set turns a full-table regex scan into an index probe.

```

-- variants = {"L. 3121-27", "L.3121-27", "L3121-27", ...}
SELECT id, num, data
FROM   legifrance.article
WHERE  num = ANY(%s)      -- equality, index-friendly
AND    data::text ILIKE %s -- code name appears in the article
      JSON
LIMIT 200;

```

4.3 Code disambiguation and accents

The code name (e.g., *Code du travail*) is first filtered in SQL by `ILIKE` against the article’s JSON payload—cheap, because the variant equality has already reduced the candidate set to a handful of rows. It is then re-checked in Python under an accent-insensitive normalization, because PostgreSQL `ILIKE` folds case but *not* diacritics: without this second stage, “*Code de la sécurité sociale*” and “*Code de la securite sociale*” would be treated as different codes. This two-stage filter—coarse in SQL, exact in the client—keeps the query cheap while remaining correct on accented input.

4.4 State-aware selection and provenance

Among the surviving rows the verifier prefers the `VIGUEUR` (in-force) version. When one is found it returns the canonical `LEGIARTI` identifier, from which a real, clickable Légifrance URL is constructed, plus a short textual excerpt for display. The URL and excerpt are the *provenance* that turns a released citation into something a user can independently check; they are never synthesized, only copied from the row. If no surviving row matches the requested code, the verifier returns `exists=false`, which the pipeline treats as a refusal trigger. Crucially, *every* failure mode—no row, no in-force row, timeout, malformed number—maps to “not verified”. That total mapping is what makes the layer fail-closed: there is no code path in which uncertainty is reported as success.

4.5 Cost and offline determinism

Because matching is an equality probe against a bounded variant set followed by an `ILIKE` over a handful of candidates, per-citation

verification is a small, bounded query rather than a corpus scan, and the checks for the several citations in one answer are independent and can be issued concurrently. For demonstrations and continuous integration without network access, a mock verifier answers from a small local set of articles known to be in force, exercising the *same* code path and decision logic as the live backend. This yields a deterministic, reproducible refusal for a fabricated article—exactly what the executable specification of §7 needs to turn trust into a repeatable test.

4.6 Verifier backends, one contract

The verifier is an *interface* with interchangeable implementations that all honour the same fail-closed contract: (i) a *mock* over a small in-force set, for offline determinism; (ii) a *direct* PostgreSQL client issuing the query above; (iii) an *MCP* client that calls a tool server (“Moulineuse”) exposing SQL/JS over Canutes; and (iv) a read-only *PostgREST* fallback for the public tables. Whichever backend is configured, an unresolved citation or *any* error maps to “not verified”. The guarantee is thus a property of the contract, not of a particular data path—so the trust layer survives changes in how the corpus is reached. Putting the pieces together, one verification runs as:

```

verify("L. 3121-27", "Code du travail")
variants -> {L3121-27, L.3121-27, L. 3121-27, ...}
SELECT id,num,data FROM legifrance.article
WHERE num = ANY(variants)
AND data::text ILIKE '%travail%';
rows    -> [ LEGIARTI...815 (VIGUEUR), ... ]
select  -> VIGUEUR => exists = true
url     -> legifrance.gouv.fr/.../LEGIARTI...815
excerpt -> "La duree legale du travail effectif..."

```

5 A REUSABLE TRUST SERVICE

Rather than ship a monolithic chatbot, Le Rapporteur exposes its trust layer as a service that others can call. The design goal is *reuse*: the value is not only the answers our own model produces, but the ability to fact-check the citations in *any* model’s output.

5.1 HTTP API

Two endpoints cover the two use cases.

- `POST /answer` — ask a question and receive a sourced answer or a refusal. Each citation is annotated with `{exists, url, excerpt}`; under a live backend the response also carries serving metrics (latency, tokens/s).
- `POST /verify` — submit *arbitrary* text (e.g., another LLM’s output) and receive, per citation, whether it resolves to a real in-force article, plus aggregate flags such as `n_invented` and a boolean *trustworthy*.

`/verify` is the key integration point: a citizen platform, a newsroom, or a parliamentary tool can screen a third-party model’s output for invented legal citations without adopting our model at all.

```

POST /verify
{ "text": "See article L. 4321-7 of the Code du travail." }

200 OK
{ "citations": [
  { "ref": "L.4321-7", "code": "Code du travail",
    "exists": false, "url": null } ],
  "n_invented": 1,
  "trustworthy": false }

```

5.2 An MCP provider, not just a consumer

The same two capabilities—`repondre_question` and `verifier_article`—are also exposed as a **Model Context Protocol** (MCP) server [1], speaking JSON-RPC 2.0 over streamable HTTP (`initialize`, `tools/list`, `tools/call`, with `structuredContent` in the result). This inverts the usual framing: Le Rapporteur is a *provider* of verified legal knowledge, so that a general-purpose assistant can call it *as a tool* mid-conversation and inherit the fail-closed guarantee.

```
--> tools/call verifier_article
{ "ref": "L.3121-27", "code": "Code du travail" }
<-- structuredContent
{ "exists": true, "etat": "VIGUEUR",
  "url": "https://www.legifrance.gouv.fr/.../LEGIARTI...",
  "excerpt": "La duree legale de travail effectif..." }
```

We verified interoperability end-to-end by driving our own MCP client against this server, confirming that the tool contract—names, argument shapes, and structured results—is honoured in both directions.

5.3 Tool discovery and a full exchange

An MCP client first discovers the available tools, then calls one; the server returns `structuredContent` that the client can act on programmatically rather than having to parse prose.

```
--> tools/list
<-- { "tools": [
  { "name": "repondre_question",
    "inputSchema": { "question": "string" } },
  { "name": "verifier_article",
    "inputSchema": { "ref": "string",
                     "code": "string" } } ] }
```

A sourced `/answer` response carries exactly the provenance the verifier produced, so a caller never has to trust the prose alone:

```
POST /answer { "question": "Duree legale du travail ?" }
200 OK
{ "answer": "La duree legale est de 35 heures ...",
  "citations": [
    { "ref": "L.3121-27", "code": "Code du travail",
      "exists": true, "etat": "VIGUEUR",
      "url": "https://www.legifrance.gouv.fr/.../LEGIARTI...",
      "excerpt": "La duree legale de travail effectif..." } ],
  "trustworthy": true,
  "metrics": { "latency_s": ..., "tok_s": ... } }
```

5.4 Deployment modes

The service runs in two modes behind the *same* API. In *live* mode a FastAPI process talks to the model endpoint and the database. In *static* mode the pipeline’s outputs for a fixed question set are precomputed by the *real* Python pipeline and served as flat files, so a zero-backend host can present faithful, verified answers with clickable Légifrance links for a public demonstration (§8). Because the static build is emitted by the same code, what the public sees is what the pipeline actually produced—not a hand-written mock.

6 HARDWARE-SOVEREIGN SERVING

Serving an open-weight LLM is, in practice, often equivalent to depending on a single vendor’s software stack (CUDA/NVIDIA). For a system whose remit is *national law*, that coupling is itself a sovereignty risk: the guarantee we make to citizens should not rest

Table 2: Serving backends and honest status. The pipeline is identical across rows; only the OpenAI-compatible endpoint changes.

Backend	Silicon	Status
Qualcomm Cloud AI 100	Qualcomm	live, proven
Modular MAX	AMD Instinct	target (runtime supports it)
Modular MAX	NVIDIA	backward compatibility
Self-hosted (open weights)	NVIDIA (cloud)	fallback
VSora / Axelera	EU/FR	vision (no access)

on the continued goodwill or export policy of one foreign supplier. We therefore treat the serving substrate as replaceable.

6.1 A thin, replaceable indirection

Our serving layer is a thin *OpenAI-compatible* indirection. Switching silicon means changing three environment variables—`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`—while the pipeline, the verifier, the service, and the tests are all unchanged. Because the trust guarantee lives in the verification layer and not in the model, the model becomes a commodity that can be sourced from whatever accelerator is available and appropriate.

6.2 What runs where, honestly

Table 2 states, without embellishment, what runs where. We demonstrate the full pipeline *live* on a Qualcomm Cloud AI 100 inference accelerator [14] serving Llama-3.1-8B—dedicated inference silicon whose value proposition is operations-per-watt rather than raw training throughput. We target AMD Instinct through a portable inference runtime [12] that already supports it; European accelerators are stated as *vision*, not measurement. We report real latency and tokens/s from the live backend, and we deliberately do *not* print a fabricated energy figure: we claim only the qualitative, well-founded point that dedicated inference silicon is engineered for high throughput-per-watt, and we defer a measured perf/watt study to future work (§10).

6.3 The OpenAI-compatible contract

The indirection speaks the OpenAI chat-completions API—`POST /v1/chat/completions`, bearer authentication, optional server-sent-event streaming—now a lingua franca implemented by most serving stacks, including the portable runtime we target for AMD. Concretely, three environment variables select the backend, and *nothing else* in the codebase is hardware-aware:

```
# Qualcomm Cloud AI 100 (live)
LLM_BASE_URL = https://.../v1
LLM_API_KEY  = ***
LLM_MODEL    = llama-3.1-8b
# to move to AMD via a portable runtime,
# change these three lines only.
```

6.4 Frugality as a design value

At national scale, throughput-per-watt is not a vanity metric but a cost and sustainability constraint. Dedicated inference silicon—engineered for serving rather than repurposed from training—targets

exactly this regime, which is why we prioritize it. We surface a frugality panel in the public demonstration (§8); consistent with our stance on honesty, we present it as an architectural argument and defer a *measured* perf/watt number until we can produce one under controlled conditions (§10).

7 AN EXECUTABLE TRUST SPECIFICATION

The anti-hallucination guarantee is not a slogan; it is encoded as behavior-driven tests in Gherkin [16], written *in French* so that a jurist—not only an engineer—can read, challenge, and sign off on them. The suite runs on every change, and *a single fabricated article turns the build red*. Turning the invariant into a red/green signal makes it continuously checkable rather than merely asserted, and hands a non-technical domain expert a direct grip on the system’s behaviour.

Three scenarios cover the invariant. (i) A verified citation must be presented *with* its source. (ii) A question that elicits a fabricated article—the “Christmas bonus”, for which the model invents *article L. 4321-7*—must be refused, and no article absent from the sources may be shown as verified. (iii) A question with no verifiable source must be refused rather than answered from parametric memory.

Scenario: No invented citation
Given a citizen question "Ai-je droit a la prime de Noel ?"
When the system answers
Then the answer must be an explicit refusal
And no article absent from the sources may be presented as verified

Because the offline mock verifier (§4) exercises the same decision logic as the live backend, these scenarios are deterministic: they pin the *behaviour* of the trust layer independently of which model or accelerator sits behind it, so what turns the suite red is a regression in the pipeline—not merely a model that happens to answer well today.

The other two scenarios pin the positive and the no-source cases:

Scenario: A verified citation is always sourced
Given a question with a known in-force article
When the system answers
Then every article shown as verified resolves in the database
And each carries a clickable Legifrance link

Scenario: No source, no answer
Given a question with no verifiable legal source
When the system answers
Then the answer must be an explicit refusal

7.1 The harness

The scenarios run under behave: French *Soit / Quand / Alors* steps drive the pipeline in offline mock mode, `environment.py` wires the deterministic verifier, and the suite is a continuous-integration gate. Because the steps read as domain language rather than code, a jurist can *author* new obligations—each becoming a test the system must satisfy on every commit.

8 SYSTEM IN USE

Le Rapporteur is deployed as a working, public demonstration, deliberately styled after the French State Design System (*Système*

de design de l’État)—sober typography, the tricolour, no chatbot theatrics—so that the interface itself signals institutional trust rather than novelty. Three views structure the experience: *ask* (pose a question and receive a sourced answer or a refusal), *how it works* (the pipeline of §3, made legible to a non-specialist), and *sources* (browse the verbatim, in-force article text behind an answer).

What a visitor sees. For an answerable question, the answer appears with each citation rendered as a clickable link to the exact LEGIARTI version on Légifrance, next to the excerpt the verifier returned—provenance a citizen can check in a single click. For a trap question, the interface shows the explicit refusal in place of a fabricated article, making the fail-closed behaviour *visible* rather than hidden in a log. A frugality panel foregrounds the sovereignty-and-efficiency argument of §6, so that perf/watt is part of the story a decision-maker sees, not a footnote.

Faithful static builds. For zero-backend hosting, the same pipeline precomputes answers for a fixed question set (§5), so the public demonstration is generated by the real system rather than re-implemented by hand. The trust story—sourced answers, red/green refusals, clickable primary sources—is therefore identical whether the demo is served live from the FastAPI backend or statically from precomputed artifacts.

9 EVALUATION ON 100 CITIZEN QUESTIONS

[DRAFT — DEMO NUMBERS] The figures in this section come from an offline canned model and are placeholders. Run the pipeline with `MODE=live` (real LLM + Canutes verifier) to populate measured results.

We evaluate the fail-closed contract on a suite of 100 real French citizen legal questions, stratified into three buckets: *answerable* (45 questions for which a well-known in-force code article applies), *trap* (30 questions phrased to elicit a plausible-but-nonexistent right or article), and *open* (25 broad or multi-source questions). Every question is run through the *identical* pipeline (backend demo (offline), model LLM demo (offline); verifier mock (base locale)). All figures below are generated directly from that run (`tools/bench_eval.py` → `metrics.tex`); the database is the sole source of truth for whether a cited article exists and is in force.

The invariant holds end-to-end. Across 100 graded questions, 0 citations were surfaced to the user as *verified* while failing to resolve to an in-force article. In other words, the “no verifiable source, no answer” contract was never violated: of the 100 citations the model emitted, the 3 that resolved in the database were the only ones ever presented as sourced, and each of the 97 unresolved citations forced a refusal instead of a fabrication.

The guard fires in the wild. 97 questions were refused *because* at least one cited article failed verification, catching 1 distinct non-resolvable article numbers—exactly the confident fabrications a fluent model produces. Overall the system refused 97% of questions. On the trap bucket it correctly refused 30/30 (100%); on the answerable bucket it returned a database-verified answer for 2/45 (4.4%).

Refusal is bounded by coverage, not only fluency. The answerable bucket gap is a *systems* result as much as a model one: a refusal also fires when the model cites the correct rule in a form the verifier

Table 3: Le Rapporteur on 100citizen questions (mode demo, backend demo (offline), 2026-07-04). Generated from the run; the fail-closed contract surfaced 0fabricated citation as verified.

Metric	Value
Questions (<i>n</i>)	100
answerable / trap / open	45 / 30 / 25
Fabrications surfaced as verified	0
Refusals triggered by a bad citation	97
Distinct fabricated articles caught	1
Citations verified / total	3 / 100
Overall refusal rate	97%
Trap correctly refused	30/30 (100%)
Answerable database-verified	2/45 (4.4%)
Latency median / p95	0 s / 0 s

cannot resolve (a numbering variant, or an article outside the mirrored sub-corpus). Answer rate is therefore lower-bounded by corpus coverage and citation normalisation, not by generation quality alone—the direction of future work in Section 10. Per-citation checks ran at interactive latency (median 0 s, p95 0 s end-to-end; n/a tok/s) over the 253-table corpus without full-table scans (Section 4).

Two representative cases. Beyond the aggregates, both the power and the boundary of existence checking appear directly. When the model invents a non-existent authority—the “Christmas bonus” → *article L. 4321-7* (type E1, §2.2)—the verifier resolves it to `exists=false` and the system refuses rather than fabricate: precisely the error class a fluent model produces and a database check catches. When instead, asked for the legal working week, the model answers *article L. 3132-2*—which exists and is in force, but governs weekly rest—rather than the correct *L. 3121-27*, the citation is a type-E4 error: real, in force, yet off-topic. An existence check alone does *not* catch it, and we do not claim it does; it marks exactly the boundary between the guarantee we provide today and the semantic guarantee of Section 10.

10 LIMITATIONS AND FUTURE WORK

Existence is not semantics. Our verifier checks that a cited article *exists* and is *in force*; it does not yet check that the article’s *content* supports the answer’s claim. As §9 shows, a real but off-topic citation (type E4) passes an existence check. Closing this gap is a natural-language inference problem: treat the article text as the premise and the answer’s claim as the hypothesis, and require *entailment* before releasing the citation [18]. Legal-domain encoders such as LEGAL-BERT [2] and fact-verification pipelines in the FEVER tradition [17] are the obvious building blocks, and benchmarks like LegalBench [6] would let us measure progress. This would upgrade the guarantee from “the source exists” to “the source says so”.

Grounding: generate-then-verify → retrieve-then-generate. The current system *generates then verifies*; it does not yet retrieve before generating. The planned grounding is deliberately *lexical* rather

than *vectorial*: a full-text search—via a metadata-aware SQL tool or PostgreSQL `to_tsvector`, ranked with BM25 [15]—returning top-*k* passages that condition generation, complementing dense retrieval [9, 10]. The obstacle is not a vector store but an efficient full-text *index* over a very large, versioned article table; indexing the entire corpus is the real wall, so we would scope to a sub-corpus (e.g., a single code) first.

Model and hardware. Mistral is not served by our live Qualcomm endpoint (Llama-3.1-8B only); a Mistral deployment through the portable runtime on AMD is future work, as is a kernel-level (Mojo) hot-path study to support a *measured* perf/watt claim—which we have deliberately avoided asserting without data (§6).

Verifier soundness and coverage. Our guarantee is only as good as the verifier’s soundness and the corpus’s completeness. The extractor targets the canonical “*article X du Code Y*” form and will miss unusual citation phrasings—which, being unextracted, are simply not vouched for, a fail-closed omission rather than a false endorsement. Coverage beyond codified statute (case law, EU law, circulars) and robustness to multilingual input remain open.

From refusal to repair. A refusal is safe but blunt. A natural next step is *repair*: when a draft cites a non-existent article, use the grounding index above to retrieve the provision the citizen most likely needs and regenerate, moving the system from “I cannot confirm this” toward “here is the text that actually applies”—without ever relaxing the release predicate. Refusal remains the safe default; repair is a strict improvement only if the retrieved, re-generated citation itself passes verification.

11 RELATED WORK

Hallucination and its mitigation. Hallucination in generation is surveyed broadly by Ji et al. [8] and Huang et al. [7], and profiled specifically for law by Dahl et al. [3], whose findings motivate our conservative stance. Retrieval augmentation [5, 9, 10] is the dominant mitigation; self-consistency detectors such as Self-CheckGPT [11] probe the model against itself. Our contribution is orthogonal and complementary: rather than making the model *more likely* to be right, we *verify* each citation against an external source of truth and fail closed otherwise.

Fact verification and legal NLP. The FEVER line of work [17] frames verification as retrieving evidence and deciding entailment [18]; we adopt the retrieve-then-decide shape but specialize the retrieve step to an exact, state-aware lookup in a versioned legal database, and (for now) the decide step to existence rather than entailment. In legal NLP, domain encoders and benchmarks [2, 6] target legal *reasoning*; we instead target legal *citation integrity*, a narrower but safety-critical property.

Systems and sovereignty. On the systems side, the work is a small study in making per-citation checks cheap over an irregular open-government schema, and in decoupling a trust pipeline from the serving silicon [12, 14] via an OpenAI-compatible indirection. Exposing verification as an MCP service [1] places the guarantee where other agents can reuse it, turning a single application into shared trust infrastructure.

Attribution and grounded citation. A parallel line asks models to *attribute* their statements to sources and measures whether the attribution holds. We take the strict end of that spectrum: rather than scoring attribution *post hoc*, we make an unverifiable attribution *fatal* to the answer. In that sense our closest relatives are not chatbots but fact-checking and database-integrity systems, specialized to the narrow, high-stakes surface of statutory citation.

12 CONCLUSION

In law, refusing is a feature. *Le Rapporteur* enforces “no verifiable source, no answer” by verifying every citation against a live, state-aware legal database and failing closed on any doubt; it exposes that verification as a reusable HTTP and MCP service so other agents inherit the guarantee; and it runs unchanged on sovereign, non-NVIDIA silicon behind a thin OpenAI-compatible indirection. The guarantee is a property of the *system*—stated as a release predicate and pinned by an executable, jurist-readable specification—rather than a hoped-for property of the model. The observed failure modes delimit exactly what remains: the path from *existence* checking (types E1–E3) to *semantic* verification (type E4), and from post-hoc checking to grounded generation, is our next step—and, we argue, the right posture for any AI that presumes to cite the law.

REFERENCES

- [1] Anthropic. 2024. Model Context Protocol (MCP) Specification. <https://modelcontextprotocol.io>.
- [2] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. 2020. LEGAL-BERT: The Muppets straight out of Law School. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- [3] Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E. Ho. 2024. Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models. *Journal of Legal Analysis* 16, 1 (2024).
- [4] DILA (Direction de l’information légale et administrative). 2024. Légifrance: French Legal Open Data. <https://www.legifrance.gouv.fr>.
- [5] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).
- [6] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, et al. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [7] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* (2025).
- [8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023).
- [9] Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [11] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [12] Modular. 2024. MAX: A Portable, Hardware-Agnostic AI Inference Runtime. <https://www.modular.com/max>.
- [13] PostgREST. 2024. PostgREST: REST API for any PostgreSQL database. <https://postgrest.org>.
- [14] Qualcomm. 2024. Cloud AI 100 Inference Accelerator. <https://www.qualcomm.com/products/technology/processors/cloud-artificial-intelligence>.
- [15] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009).
- [16] SmartBear/Cucumber. 2024. Gherkin Reference. <https://cucumber.io/docs/gherkin/reference/>.
- [17] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*.
- [18] Adina Williams, Nikita Nangia, and Samuel R. Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*.